GOSIM

GOSIM Shenzhen 2026

压缩率之上：
面向分布式 LLM 推理的服务感知 KV Cache 压缩

报告人：刘泽栋 | 中国科学院计算技术研究所 · 高性能计算机研究中心

GOSIM Shenzhen 2026 · 深圳 · 10 月 17 日



CONTENTS

Part 01 Agent 带来的KV缓存与传输压力

Part 02 KVServe：核心设计与收益

Part 03 从论文到工程落地的拓展

Part 04 社区共建：欢迎参与贡献



01

Part 01 「Agent 带来的缓存问题 」

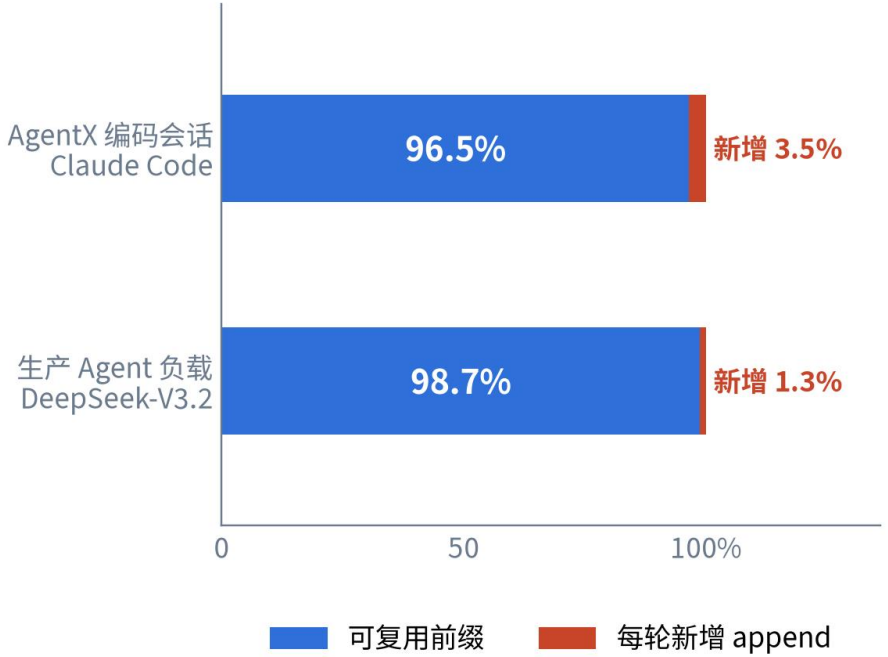


Agent 负载画像：长前缀、高复用、小增量

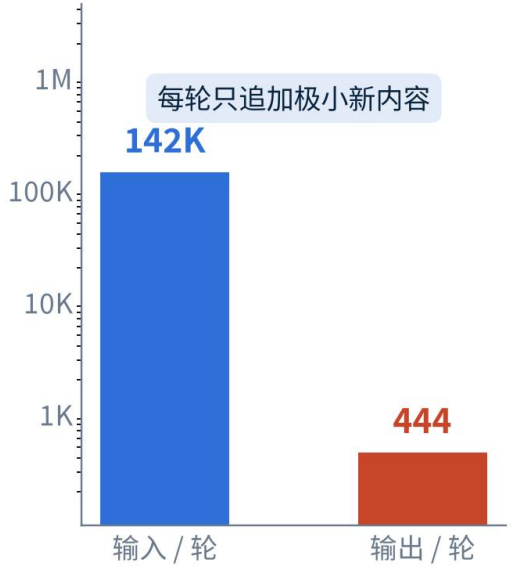
高复用说明KV 值得存下来；显存装不下，就只能分布式分层存放 —— 我们要压缩的不只是比特数，更是这条保存与搬运的链路。

数据来源：vLLM × AgentX 编码会话分析（2026-09，393 会话，中位 43 轮） · DualPath 生产 Agent 负载（arXiv:2602.21548，均值 157 轮） · vLLM Mooncake Store（2026-05）

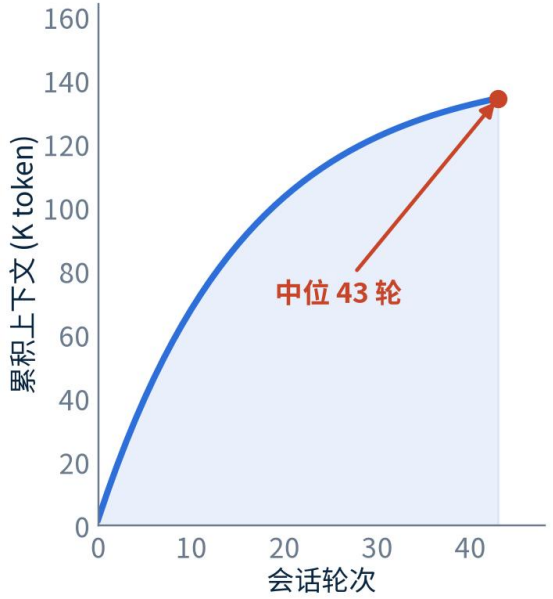
① 高复用：请求几乎全是已见前缀



② 小增量：142K 输入 vs 444 输出



③ 长前缀：上下文只涨不落



④ 会分支：44% 的会话含至少一个 subagent（这些会话平均 4 次恢复）—— 分支点决定要不要额外保留前缀快照。

① 长前缀

上下文只涨不落：会话中位 43 轮，输入中位 142K token；生产 Agent 负载均值 157 轮。

② 高复用

前缀命中 >96%（生产负载 98.7%）：同一份前缀被反复复用 —— 存下来即可免去重算。

③ 小增量

每轮仅追加 ~400–450 token：保存与取用成为新的主要成本。

KV 变重之后：分离式推理越分越细、越离越远

长上下文 + 多轮复用：PD 分离仍在继续细化、集群也越离越远，KV 的产生、搬运与存放同时进入关键路径

[1]KVServe, SIGCOMM 2026, §2.1 · [2]vLLM × AgentX（2026-09） · [3]DualPath（arXiv:2602.21548） · [4]vLLM Mooncake Store（2026-05） · [5]PDD（arXiv:2609.13161） · [6]PrfaaS（arXiv:2604.15039）

① 重算开销很大

KV 体量随 token 数线性增长：单会话输入中位 142K token，如果没法尽量复用前缀缓存，则需要重新 prefill 超长的序列。

② PD分离：越分越细、越离越远

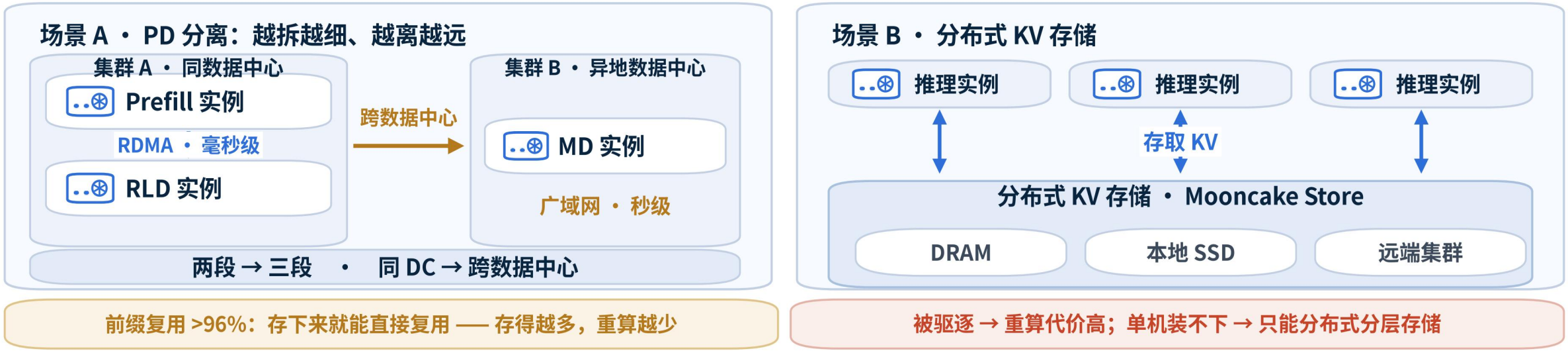
PD 分离已是标配，仍在继续细化：PDD[5] 把解码再拆一层、跨到第二个数据中心；两段计算越离越远 —— 从RDMA到**跨域、跨中心**的广域网（PrfaaS）[6]

③ 容量不足：使用分布式存储

前缀复用 >96% 说明存得越多、重算越少；单机装不下，KV 只能下沉到 DRAM / SSD 与远端集群，存取与恢复随之进入推理的关键路径。

分离式推理的两种形态：PD 分离 + 分布式 KV 存储

KV 通信最高占 JCT 的 60%

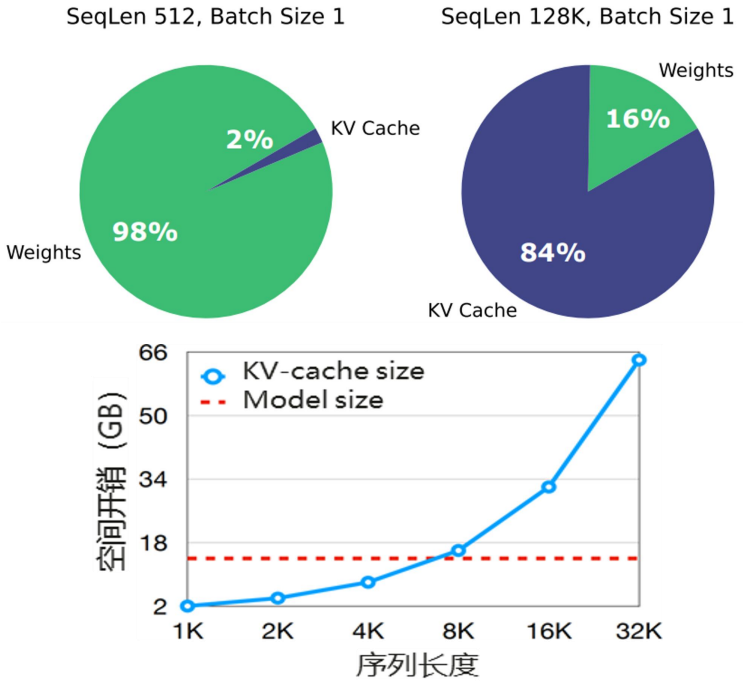


前缀复用 >96%：存下来就能直接复用 —— 存得越多，重算越少

被驱逐 → 重算代价高；单机装不下 → 只能分布式分层存储

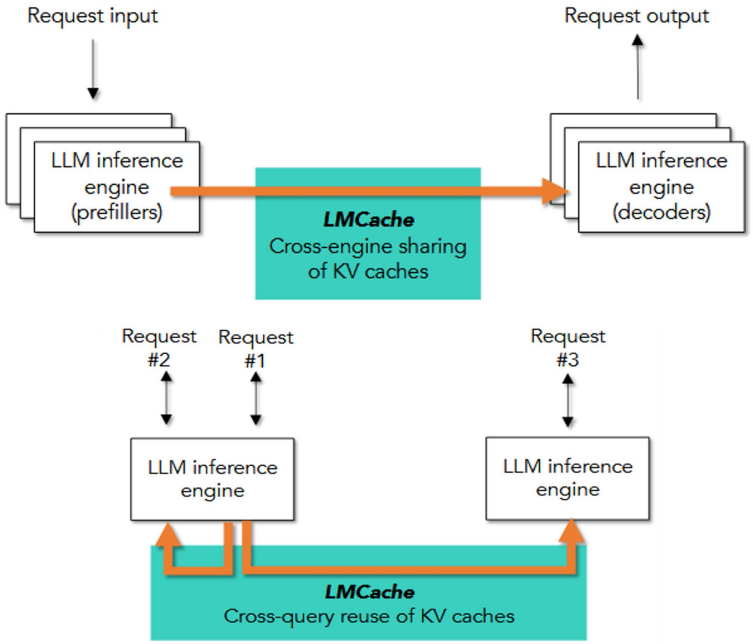
KV Cache 压缩需求强烈

上下文只涨不落



- Agent 多轮工具调用让上下文单调增长：会话中位 43 轮、输入中位 142K token
- 长上下文场景中，KV Cache 成为推理的主要显存瓶颈

跨节点 / 跨实例搬运 KV：介质决定快慢

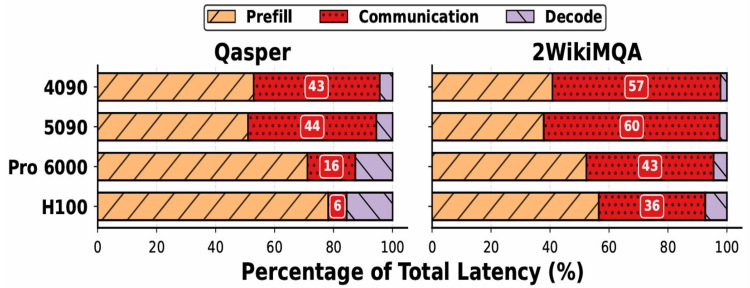


- 同 DC 内是 RDMA（微秒级）；跨域 / 跨中心只剩以太网与广域网 —— **时延高几十到上百倍。**
- 介质一换，搬运就慢到不可忽略：Mooncake Store 把 KV 池扩展到 CPU/DRAM 与磁盘分层。

KV传输与存储 I/O成为瓶颈

Table 3: KV throughput Φ_{kv} (Gbps) at various input lengths. All models are benchmarked on 8×H200 with SGLang v0.5.9 [7].

	Hybrid				Dense	
Seq Len	Kimi Linear	MiMo-V2-Flash	Qwen3.5-397B	Ring-2.5-1T	MiniMax-M2.5	Qwen3-235B
1K	1.19	0.82	4.13	7.27	4.94	4.12
8K	2.29	2.85	6.28	4.47	32.87	22.42
32K	3.87	4.66	8.25	2.59	59.93	33.35
128K	4.88	4.71	7.47	1.46	47.82	21.50



- **PD分离**：32K 输入下单实例 KV 传输需求达 **33–60 Gbps**，KV 通信最高占 JCT 的 **60%**
- **分布式KV**：生产 Agent 负载中，KV 存储节点的 I/O 也成为主要瓶颈（DualPath）

KV Cache 压缩相关工作

CacheGen：面向传输的编码

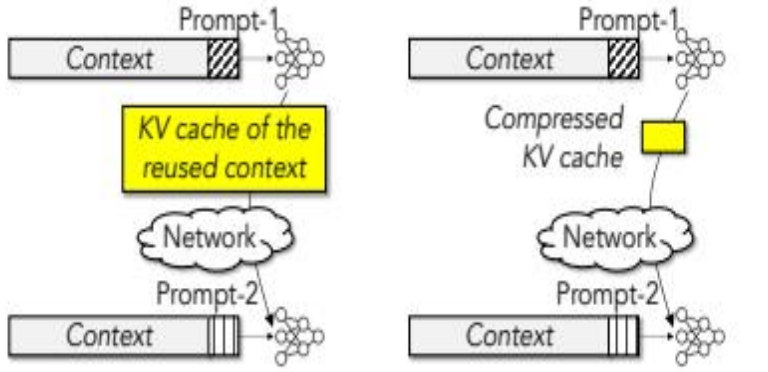
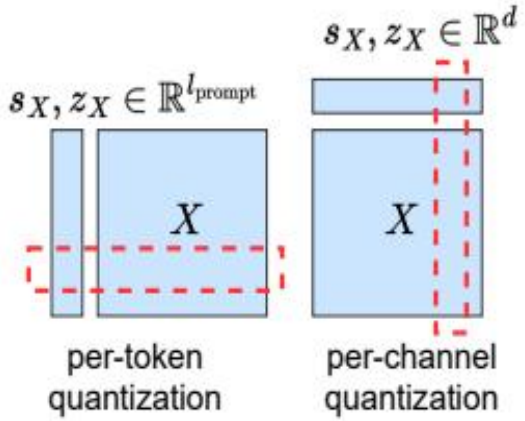


Figure 1: *When the context is reused, CacheGen speeds up the sharing of its KV cache by compressing (encoding) the KV cache.*

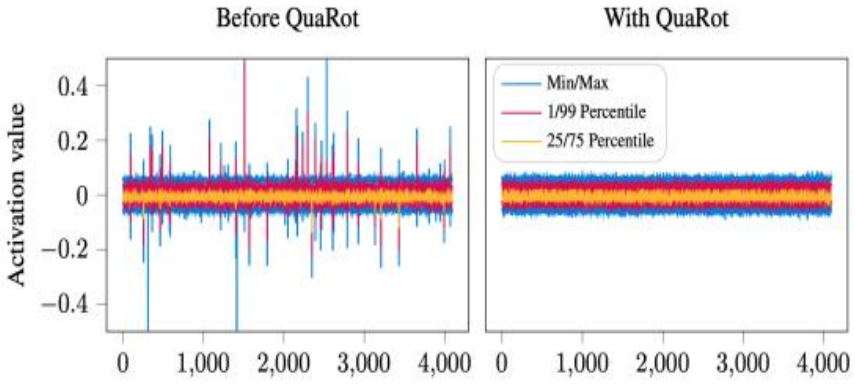
- 将 KV 编码成紧凑比特流，降低跨节点传输量。但仅按层选择混合量化精度。
- 分块流式发送，按可用带宽调整压缩级别；体积缩小 **3.5–4.3×**

KIVI：K/V 非对称 2-bit



- K Cache 存在固定通道离群值，**按通道量化**
- V Cache 则 **按 token 量化**；免调优实现 2-bit KV Cache

QuaRot：旋转后端到端 4-bit



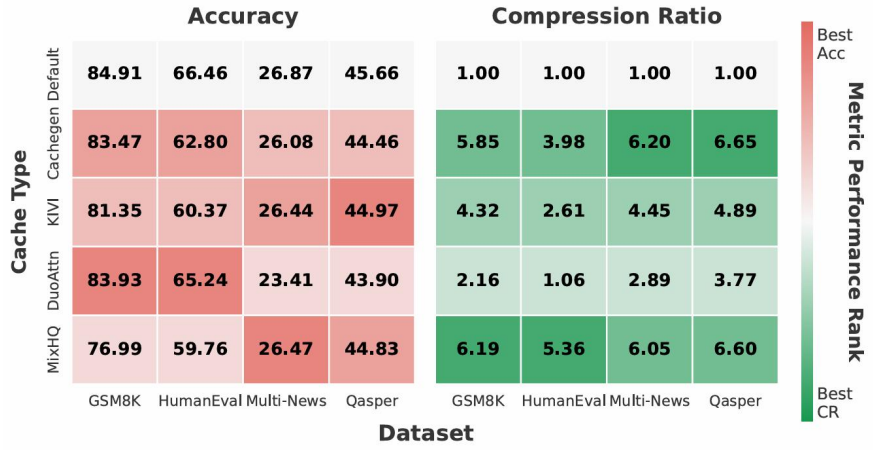
- 引入随机 Hadamard **旋转矩阵**，等价变换并分散离群值
- 权重、激活与 KV Cache 统一 **端到端 4-bit**，无需保留高精度通道

Y. Liu et al., CacheGen, SIGCOMM 2024; Z. Liu et al., KIVI, ICML 2024; S. Ashkboos et al., QuaRot, NeurIPS 2024.

7

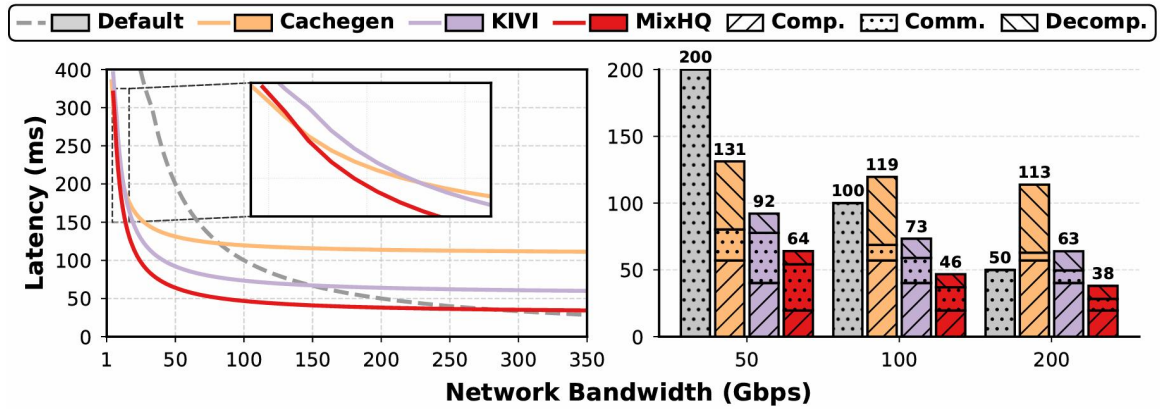
重新思考：最优KV压缩算法随工作负载与带宽变化

最优算法随工作负载变化



- 最优策略不固定：不同任务最优 KV 压缩方法不同，单一静态策略难适配多样工作负载
- 精度与压缩率均会切换：DuoAttn、KIVI、MixHQ、CacheGen 在不同数据集各有优势

最优算法随带宽变化

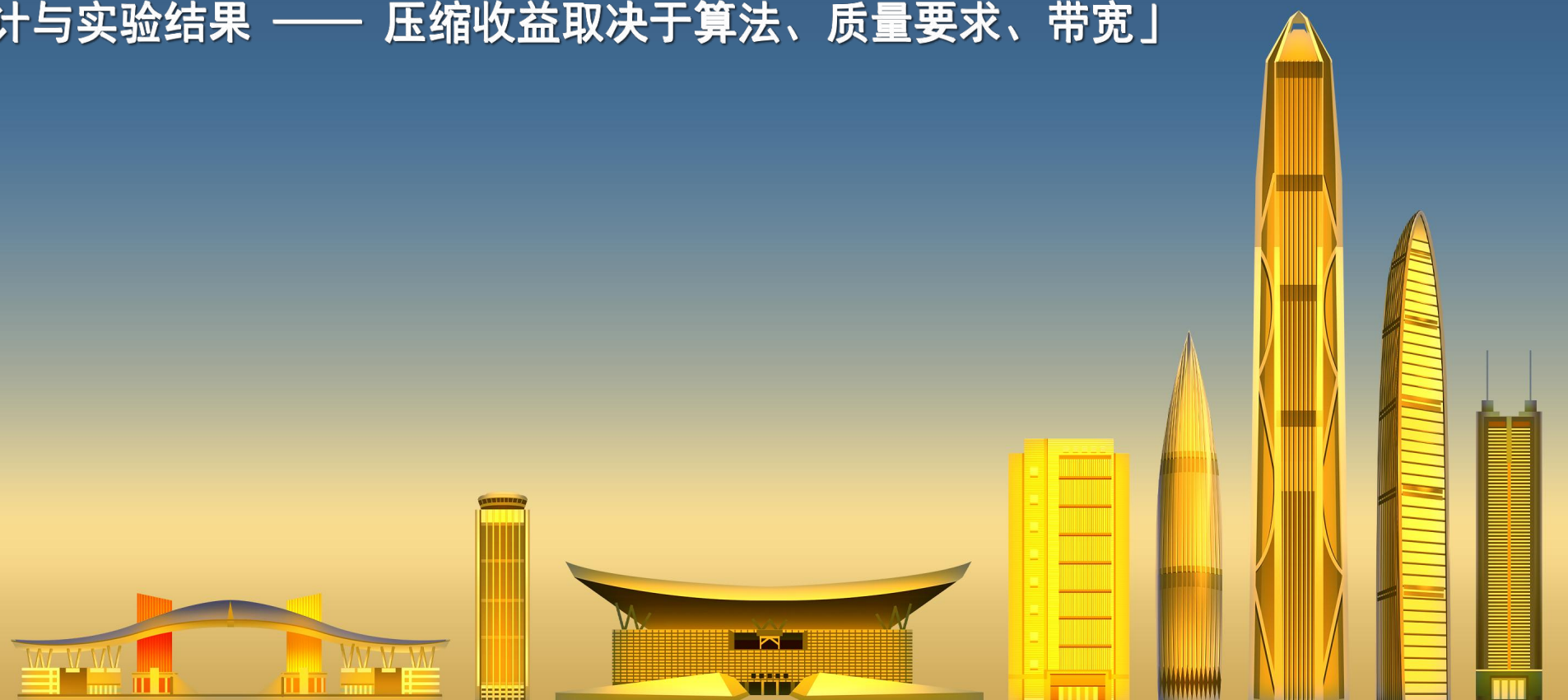


- 低带宽收益大：减少 KV 传输量显著降低延迟，明显优于 Default, 但在大于250GBps时最优压缩也无法带来收益
- 最优策略动态变化：通信占比下降、压缩/解压开销主导，最优策略发生切换，没有永远最优的策略

压缩率不是唯一目标：最优策略随工作负载与带宽双重变化 —— 决策应由服务状态驱动

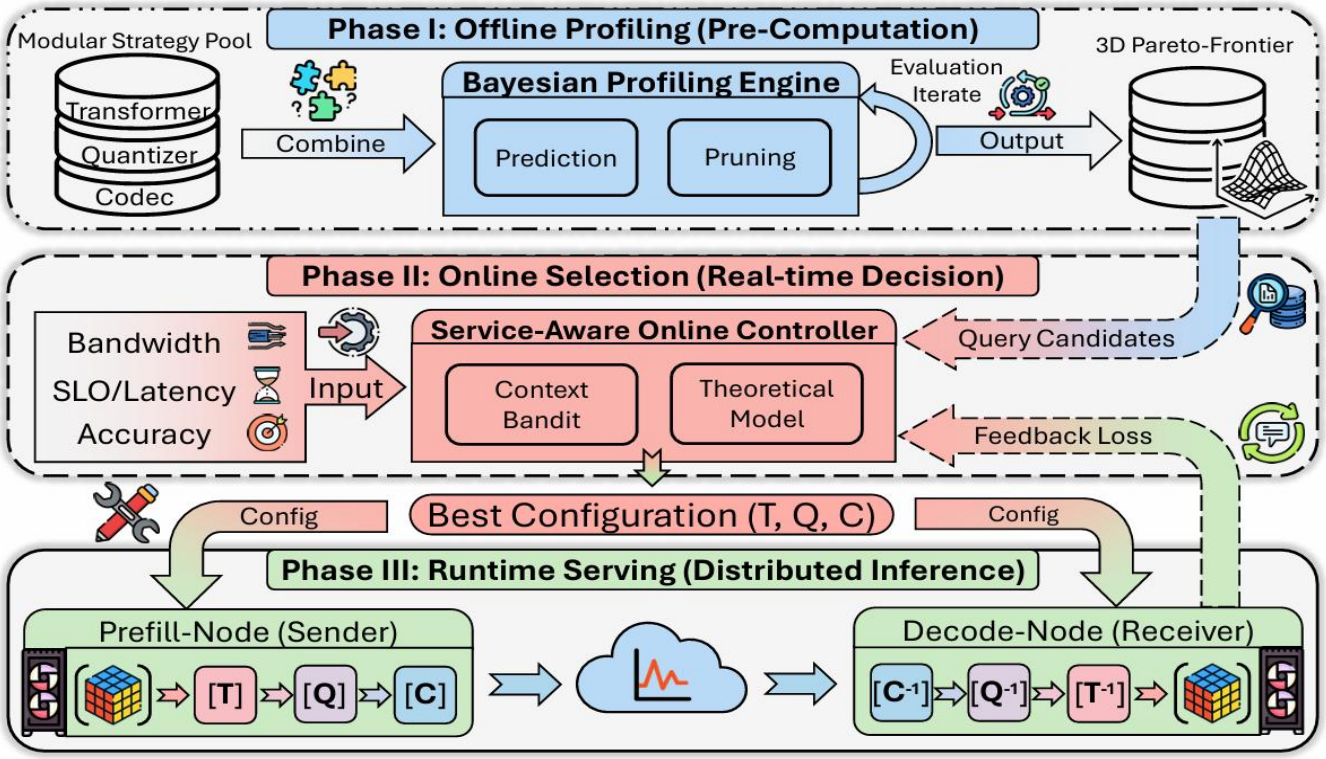
02

「KVServe 的核心设计与实验结果 —— 压缩收益取决于算法、质量要求、带宽」



KVServe：离线-在线混合决策的高效KV压缩策略

KVServe：自适应 KV 压缩框架



1. 阶段一：离线搜索

将 Transformer、Quantizer、Codec 组成模块化策略池；经 Bayesian Profiling Engine 离线搜索候选配置，形成 3D Pareto Frontier。

2. 阶段二：在线决策

结合 Bandwidth、SLO/Latency、Accuracy 等在线上下文，由 Online Controller 实时选择，并以 theoretical model 与 context bandit 快速决策。

3. 阶段三：运行时执行

Prefill 节点按选中配置执行压缩流水线 [T]→[Q]→[C]，跨节点传输压缩 KV；Decode 节点执行 $[C^{-1}]\rightarrow[Q^{-1}]\rightarrow[T^{-1}]$ 完成恢复。

核心思路：先离线搜索候选，再在线动态决策，最后在分布式推理中高效执行

KVServe：问题建模与优化目标

① 决策输入 · 服务上下文

$c = (w,\ B,\ T_{SLO},\ q_{min})$

w　会话段的工作负载类别，由上层路由器 / 分类器给出

B　当前有效带宽：网络或 I/O 的 goodput，而非标称带宽

T_{SLO}　该会话段的时延预算

q_{min}　最低质量要求

以**请求**为决策粒度：KV 搬运开始时选定策略，整个请求内保持不变。

② 决策对象 · 压缩策略

$p = (cr_p,\ s_p,\ q_p)$

cr_p $= V / V_p$　压缩率

V 为会话段内待搬运的原始 KV 字节数

s_p $= (1/s_{enc} + 1/s_{dec})^{-1}$

有效 (解) 压缩吞吐：编码与解码吞吐的调和平均

q_p　工作负载 w 下的质量（如任务精度）

三元组由离线 profiling 给出，在线只在这组参数上做约束优化。

③ 时延模型与约束

时延模型（会话段 JCT）

$$T_p(c) = T_{model}(w) + V/s_p + V/(B \cdot cr_p)$$

$$T_0(c) = T_{model}(w) + V/B$$　（不压缩基线）

三项依次是：与压缩无关的模型执行开销、压缩与解压自身开销、压缩后 KV 的通信时间。

可行集（质量和SLO约束）

$$P(c) = \{\ p \in P : T_p(c) \le T_{SLO},\ q_p(w) \ge q_{min}\ \}$$

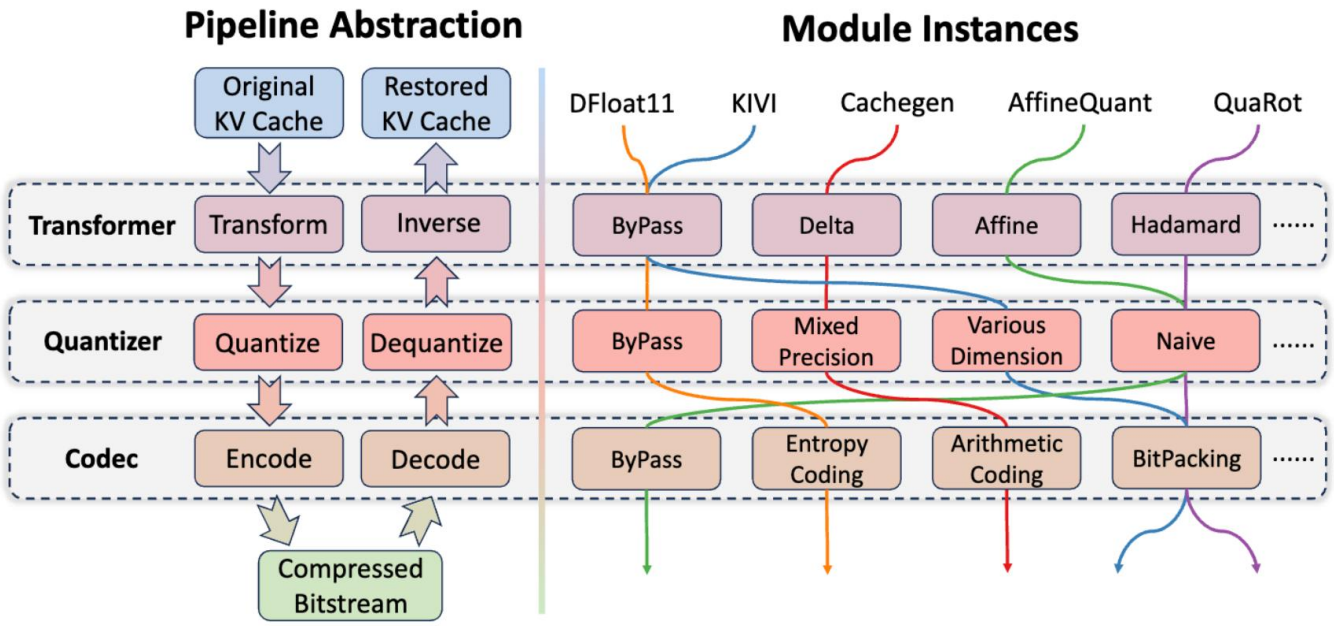
P 为全部可选策略；时延与质量必须同时满足，缺一即不可行，p*(c) 在每个会话段重新求解。

优化目标：p*(c) ∈ arg min T_p(c)　——　B 决定能省多少，s_p 决定压缩自身开销，cr_p 决定压缩后体积，q_p(w) 决定质量代价

依据：KVServe, SIGCOMM 2026 · §3（Serving System Model / Constrained Optimization）

KVServe：KV 压缩算法池

三阶段压缩流水线



1. 由 变换-量化-无损压缩 三阶段组成

将 Transformer、Quantizer、Codec 组成模块化策略池；经 Bayesian Profiling Engine 离线搜索候选配置，形成 3D Pareto Frontier。

2. 将主流 KV 压缩算法映射到算子池中

主流算法如 QuaRot，KIVI、CacheGen 等，均能拆解成对应阶段算子映射到本算法池中。也就意味着 KVServe 的搜索空间**涵盖了主流压缩算法**

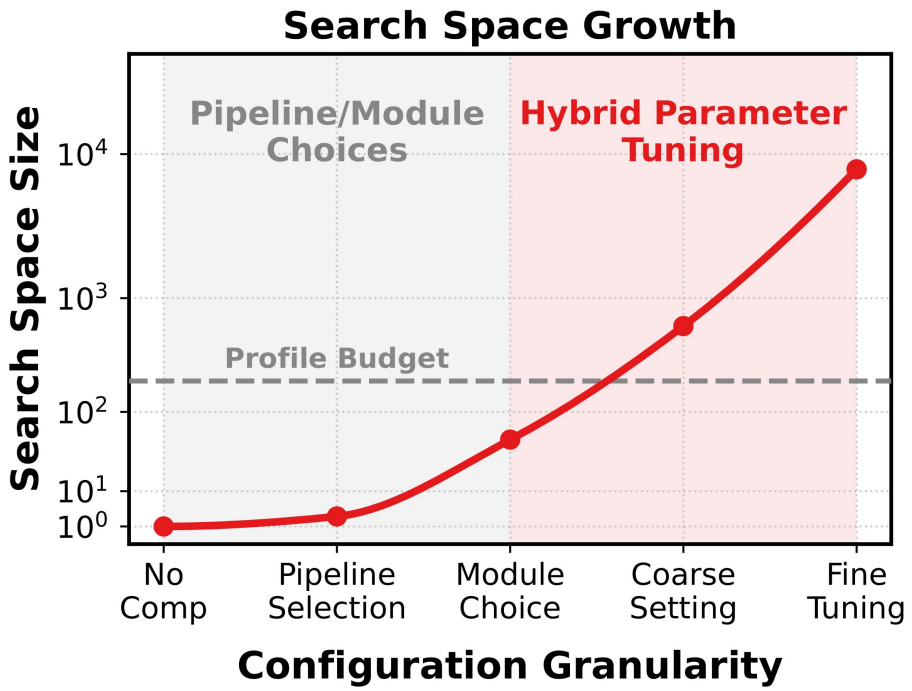
3. 提出新的 MixHQ 量化算法

发现现有量化算法缺乏 Head 维度的混合精度量化感知。补充按 Head 维度划分的量化算法 MixHQ，填补了算法搜索空间中的空缺。

将主流算法映射到模块化算法池中，各阶段算子可自由拆卸并组合，涵盖了更大的算法搜索空间

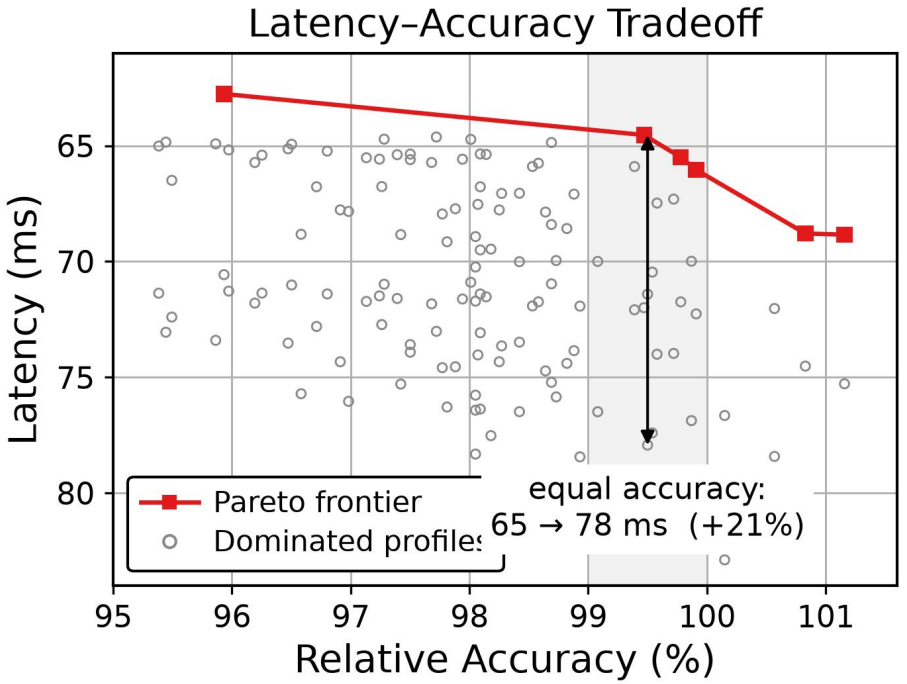
KVServe：Runtime下选择最优算法的两大挑战

挑战 1 · 策略空间的组合爆炸



候选数随配置粒度**近似指数增长**：细粒度调优即逼近 10^4 个候选，离线 profiling 预算远低于此。

挑战 2 · 时延–质量权衡缺少决策准则

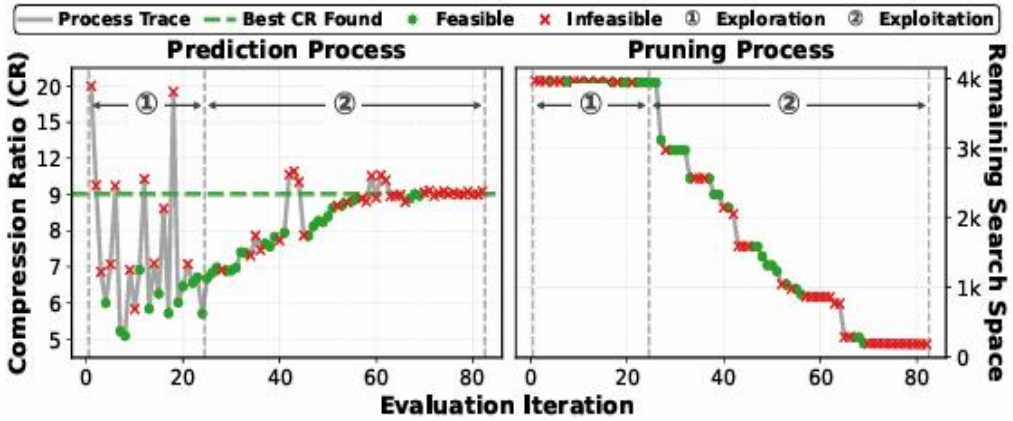


同一精度水平内，时延**相差 21%**：按质量排序分不出快慢，**按压缩率排序可能跌破质量下限。**

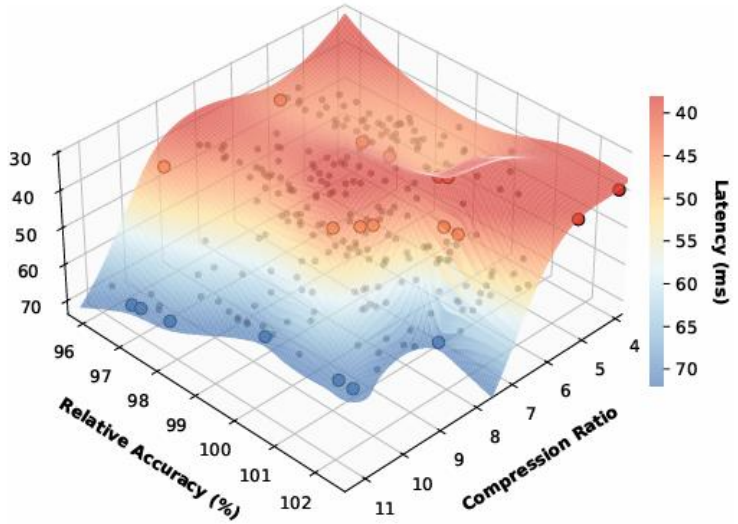
两大挑战各对应后面的一点设计：离线搜索引擎把空间搜小，在线决策器在约束下选点

KVServe：贝叶斯离线搜索引擎

搜索过程：将搜索成本降低 50x（原需 50 GPU时）



搜索结果：3D pareto 前沿



优化目标：

$$\max_{\mathbf{c}} \mathrm{CR}(\mathbf{c}) \quad \text{s.t.} \quad \mathrm{Acc}(\mathbf{c}) \geq \mathrm{Acc}_{\mathrm{threshold}},$$

统一参数编码

将离散的模块选择和连续的数值参数映射到统一向量空间，使高斯过程能够建模不同配置之间的相似性。

探索利用策略

搜索早期偏向探索高不确定性配置，后期逐步转向利用高收益配置，从而更快收敛到优质方案。

双向搜索剪枝

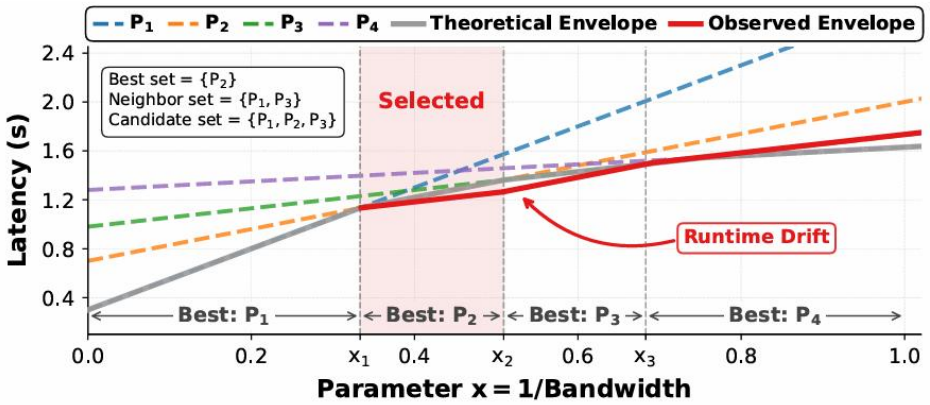
利用压缩率与精度之间的近似单调关系，剪掉明显不可行或收益较低的配置，缩小搜索空间。

自适应早停

当连续搜索失败或有效候选空间耗尽时提前终止，避免无效评估，降低离线 profiling 开销。

KVServe：服务感知的在线决策器

决策过程



在线学习模型

$$c = (w, B, T_{SLO}, q_{min})$$

$$T_p(c) = T_{model}(w) + \frac{V}{s_p} + \frac{V}{B \cdot cr_p}$$

$$B_p^\star = (1 - \frac{1}{cr_p})s_p, \quad T_p(c) < T_0(c) \Leftrightarrow B < B_p^\star$$

$$\tilde{T}_p(x) = \frac{1}{s_p} + \frac{1}{cr_p}x, \quad x = \frac{1}{B}$$

$$T_p^{eff} = \hat{T}_p(c) + \bar{\delta}_{b,i}(p)$$

服务上下文感知

根据 workload、有效带宽、SLO 和精度预算，动态选择当前请求的压缩配置。

收益阈值过滤

通过 B_p^* 判断压缩是否真正带来加速，过滤高带宽下可能造成负优化的配置。

分段最优查表

将不同 profile 的延迟表示为 x=1/B 上的直线，利用 lower envelope 实现 O(1) 在线选择。

残差 Bandit 修正

围绕理论最优 profile 及其邻居构造小候选集，用运行时观测残差修正离线模型偏差。

压缩率与推理质量评估

- KVServe 在压缩算法库中针对工作负载进行离线搜索，求解出的最优配置超越了现有的SOTA 算法

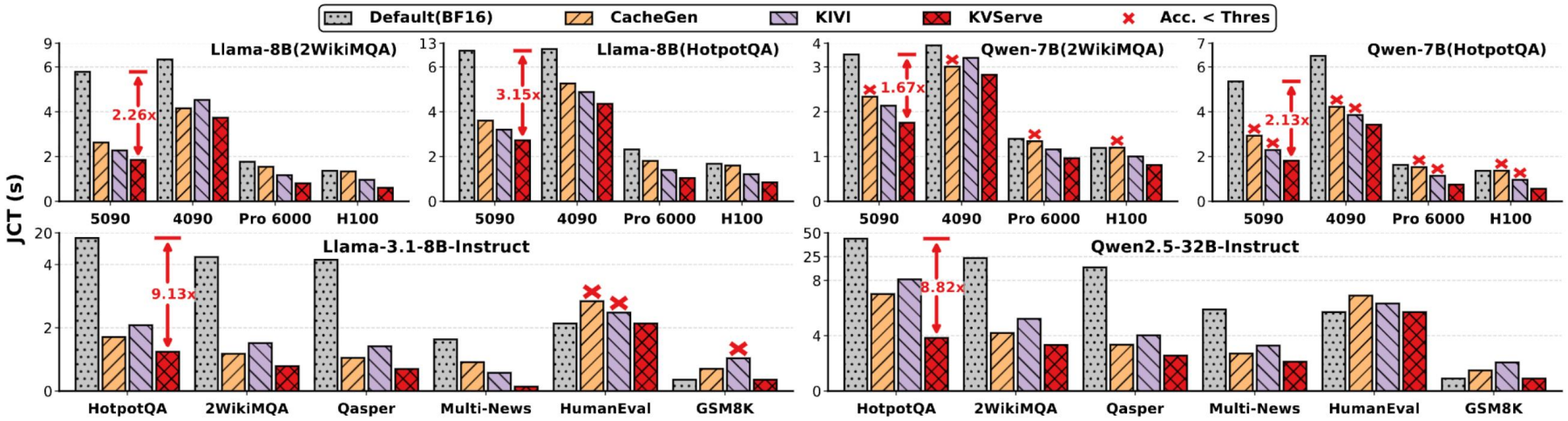
压缩率与准确率：平均准确率 100.35%，压缩率 8.28x（超 TurboQuant 6x）
在六项任务上都超越了现有 SOTA 算法，并且在四项任务上正确率超越了 bf16 基线

Method (Acc / CR)	Profiling Workloads				Unseen Workloads		Average (Rel. Acc / CR)
	GSM8K	HumanEval	Multi-News	Qasper	2WikiMQA	HotpotQA	
Default (BF16)	82.64 / 1.00	83.54 / 1.00	23.73 / 1.00	43.34 / 1.00	46.96 / 1.00	57.53 / 1.00	100.00 / 1.00
CacheGen	72.55 / 6.01	57.32 / 4.06	17.95 / 6.33	25.95 / 6.81	28.53 / 6.84	24.09 / 6.94	65.76 / 6.17
KIVI	81.50 / 4.26	81.71 / 2.49	23.38 / 4.50	41.05 / 4.96	46.33 / 5.04	55.37 / 5.15	97.43 / 4.40
DuoAttention	82.56 / 2.21	82.93 / 1.06	20.43 / 2.92	40.53 / 3.83	45.45 / 4.08	56.00 / 4.50	95.48 / 3.10
KVServe-Unified	81.50 / 7.07	**84.15 / 6.20**	23.18 / 7.36	42.35 / 7.85	**47.04 / 7.94**	54.23 / 8.07	98.20 / 7.42
KVServe-Aware	**84.53 / 7.29**	**84.15 / 6.04**	**24.75 / 10.12**	**43.48 / 8.60**	46.32 / 8.72	55.11 / 8.90	**100.35 / 8.28**

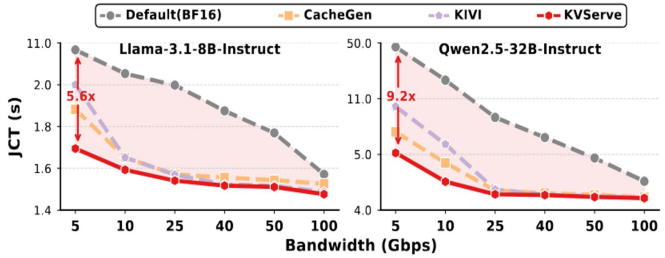
压缩率与推理性能评估

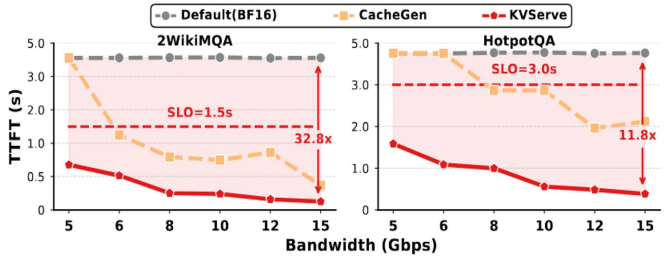
- 在Llama-8B Qwen2.5-7B Qwen 2.5-32B三种模型；RTX 4090、5090、Pro 6000、H100四种平台进行测试

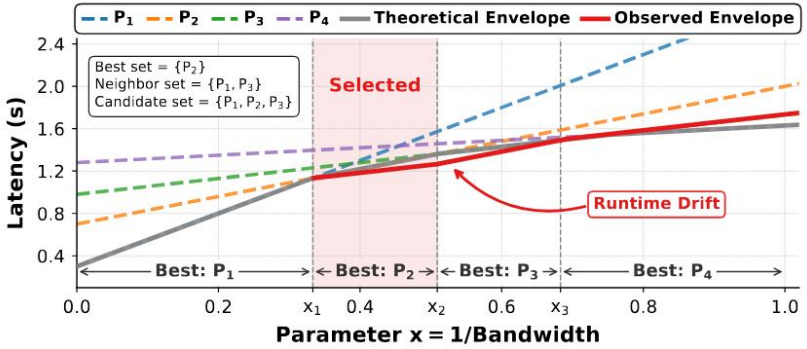
  KVServe 率先将 KV 压缩从算法选择转化为服务状态驱动的动态控制问题，在波动的带宽与负载下始终锁定最优性能边界



端到端延迟：PD 分离/前缀缓存最高加速比 9.2x 和 32.8x　　在线调优：结合理论与轻量级模型，自适应修正运行时偏差







KVServe 的核心：动态 · 可插拔 · 可组合

服务感知 KV 压缩不是一个固定配方，而是一套按服务状态决策、且可被替换与组合的算法系统

算法池 × 离线搜索 × 在线决策 · 同一套思想后面的三种落地：更快的实现、更深的存储、更开放的框架

服务感知 · 动态

决策随着服务状态变化

带宽、SLO、缓存命中率一变，最优压缩方法就变。离线搜索给候选，在线决策器持续选择，残差 bandit 修正理论模型的偏差。

证据：PD 分离 9.2× · 前缀缓存 32.8×

可插拔

算法是灵活组件，不是死代码

变换算子 / 量化器 / 编码器三段各自注册即可接入；换算法不改调度，换传输不改算法，也不动已训好的模型。

示例：Hadamard · MixHQ · nvCOMP

可组合

组合空间离线搜索

三个阶段自由组合后进入 3D Pareto 前沿，按质量与带宽约束取点，而不是挑一个固定配方。

证据：六任务平均 100.35% · 压缩率 8.28×

同一套思想的两种落地 —— 也正是接下来的两节

接下来 · 更快的算法

FastKVComp：把压缩路径做到极致

同一套压缩语义重写成更快的实现：Decode 开销仅 +0.71%，端到端 JCT 最高降低 1.62×。 → 第 17 – 18 页

接下来 · 存储层与框架层落地

KVServe-Kunpeng / Mooncake

把压缩下沉到多级 KV 缓存与共享存储池：DDR KV / rank −67.9%，Mooncake 链路 TTFT 1.53 s → 0.96 s。 → 第 19 – 22 页

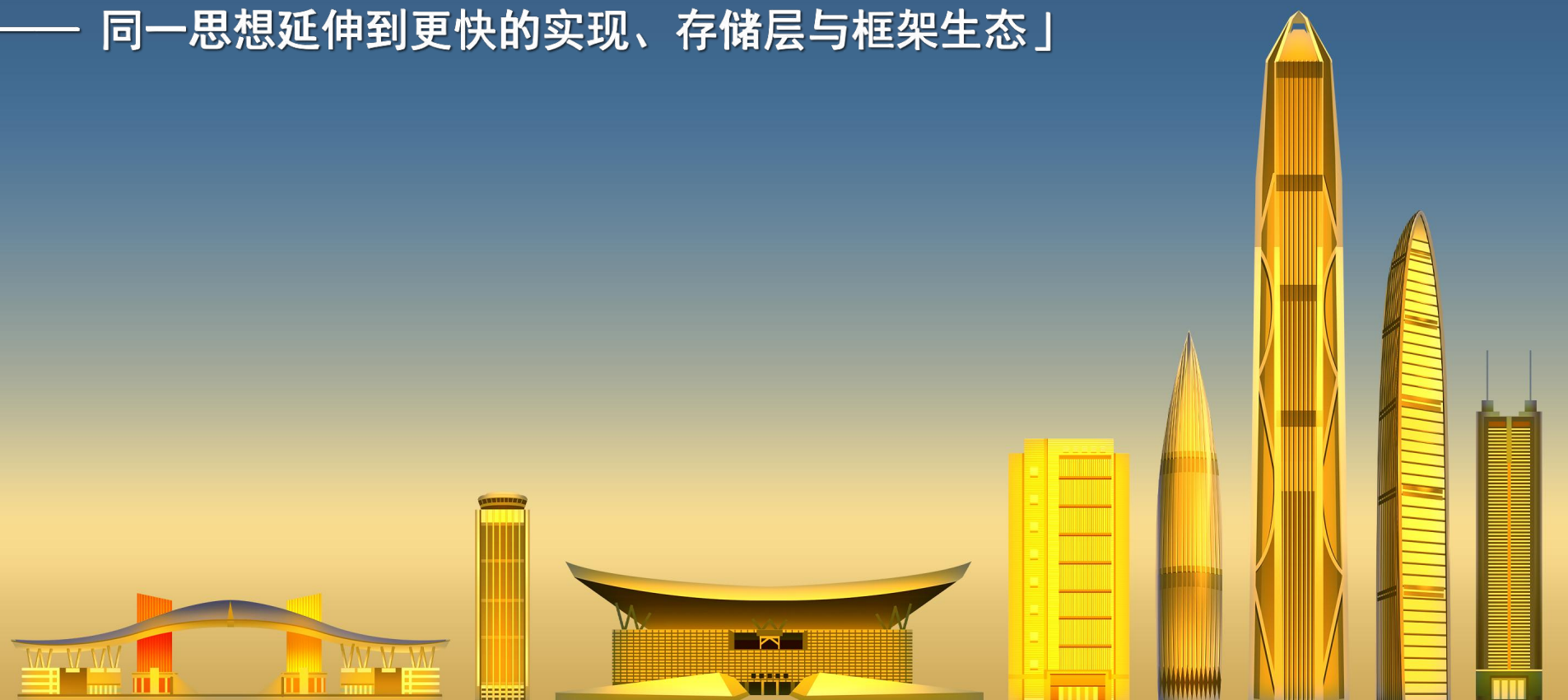
合起来才是服务感知：算法选择随服务状态变化，而算法本身随时可被替换、可被组合 —— 这既是 KVServe 的接口，也是后面两节的出发点。

GOSIM

GOSIM Shenzhen 2026

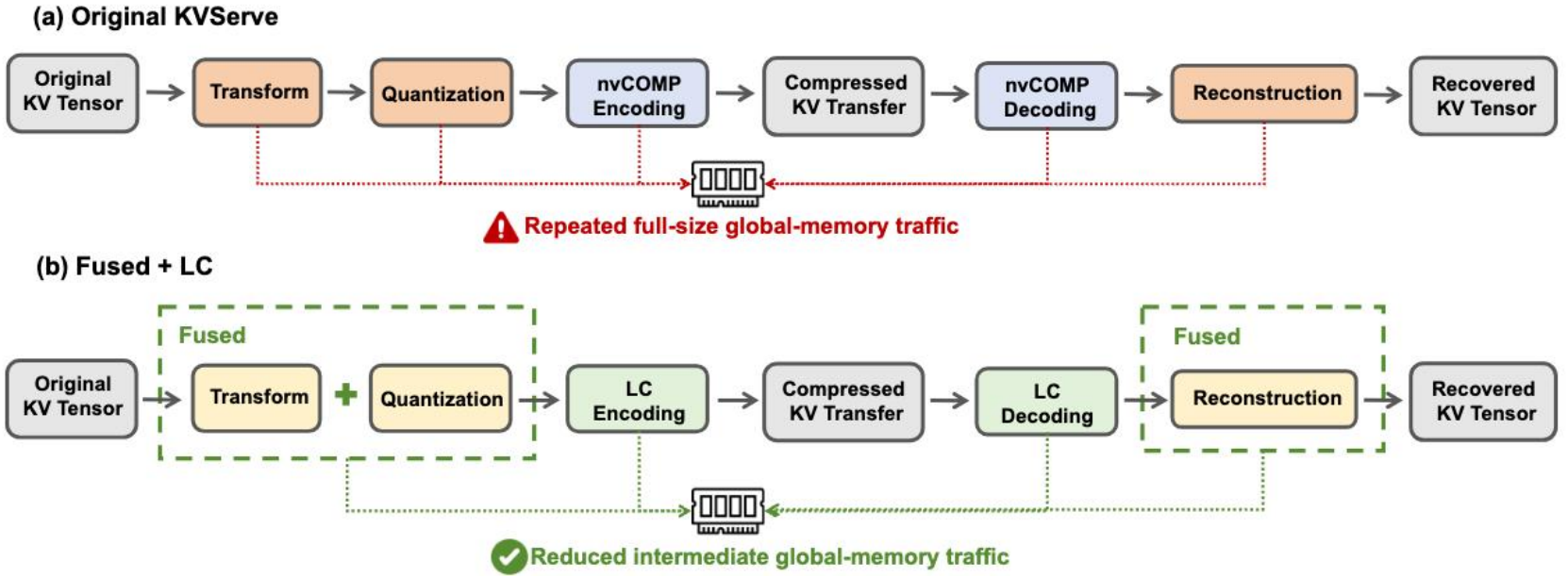
03

「从论文到工程落地 —— 同一思想延伸到更快的实现、存储层与框架生态」



FastKVComp：KV 压缩路径的极致性能优化

- KVServe的压缩路径目前仍存在性能优化机会，限制了其在更快的高速互联网络（如Infiniband）中的拓展机会



1 融合数据变换与量化操作，节省访存时间

原路径把 Transform 与 Quantization 拆成两个 GPU kernel：前者把完整中间张量写回 HBM，后者再读回后量化。中间结果只服务下一阶段，却多付一次写读，并叠加两次启动与同步开销。

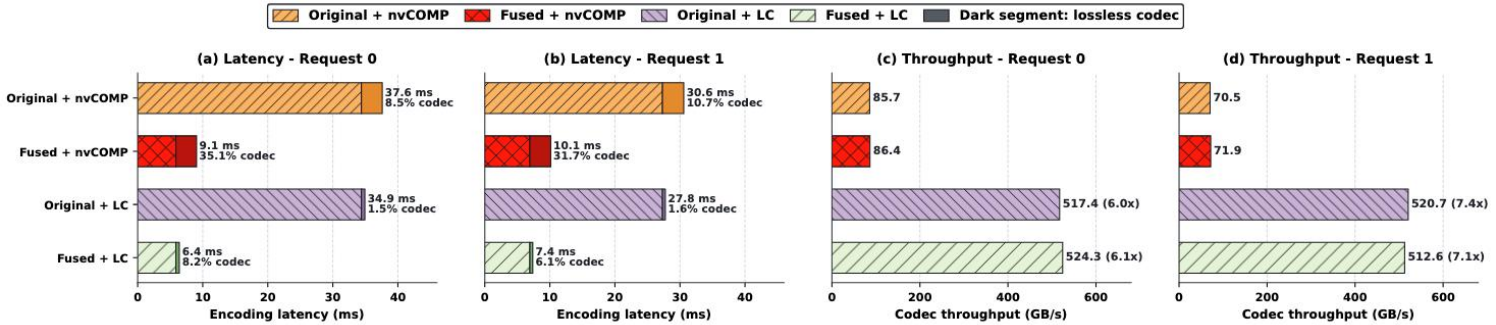
2 替换成可扩展性的无损编码后端，释放吞吐上限

原 KVServe 经既有路径调用 nvCOMP，算法选择有限，难以拓展。实测无损阶段端到端吞吐**不足 100 GB/s**，需要更可组合的高性能原语。

FastKVComp：压缩性能与端到端收益

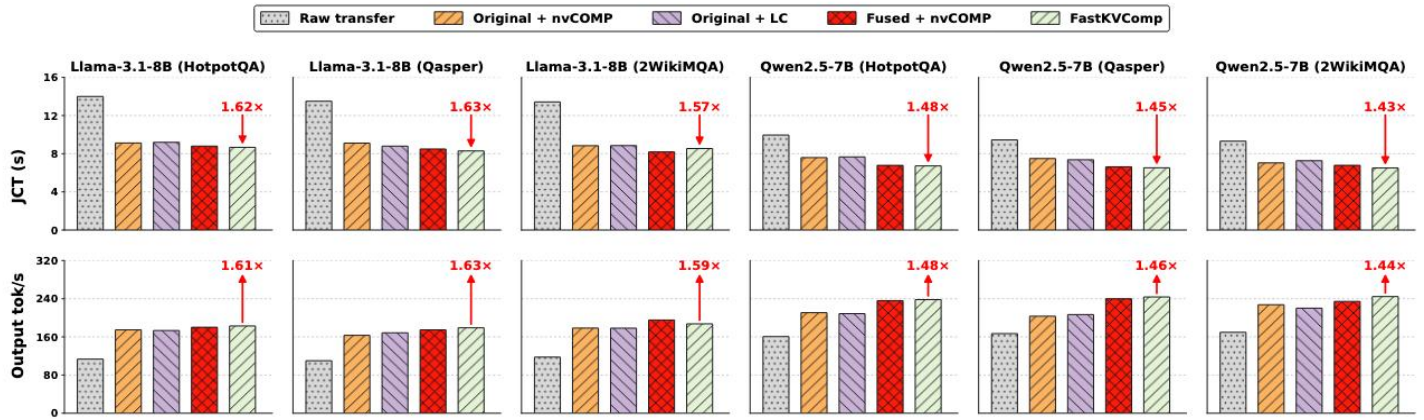
- 测试环境为 RTX 6000p 与 H100 异构节点，互联带宽为 200Gb/s IB。包含 Qwen 7B、Llama 8B 模型和三类数据集

融合 Kernel 和 LC 无损压缩性能评估



- 融合 Kernel 将变换与量化计算速度提升 **3.9～5.8x**，降低 80% 左右的延迟
- LC 无损压缩后端将压缩吞吐从 90GB/s 提升到**500+GB/s**，带来极大的性能提升

FastKVComp 端到端性能评估



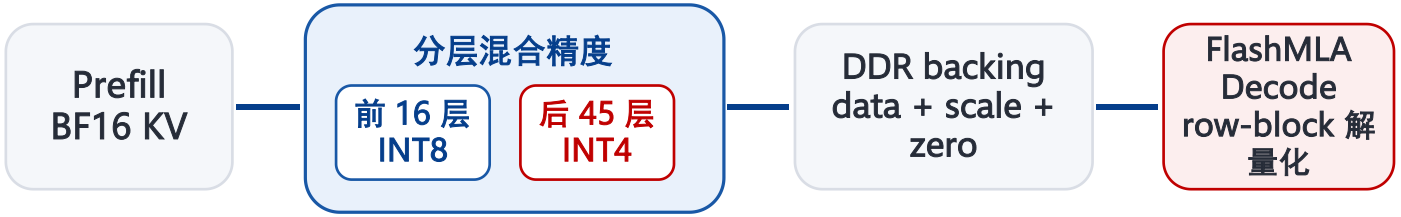
- 优化后的KV 通信压缩在 200Gb/s 的 Infiniband 互联下仍有可观的性能收益
- FastKVComp将**作业完成时间（JCT） 最高降低 1.62x，推理吞吐最高提升 1.63x**

KVServe-Kunpeng：落地深超鲲鹏平台，优化多级KV缓存

■ 优化思路：KV cache 混合精度量化，实现容量和精度的折中

端到端量化路径

Prefill 写入即压缩，cache round-trip 保持量化数据、scale / zero-point 与精度元数据一致



容量 / 性能实测（Q16）

DDR KV / rank



	BF16	Q16	开销
Prefill	1969.2	1979.7	+0.53%
Decode	1151.8	1159.9	+0.71%

Decode 吞吐 55.18 tok/s

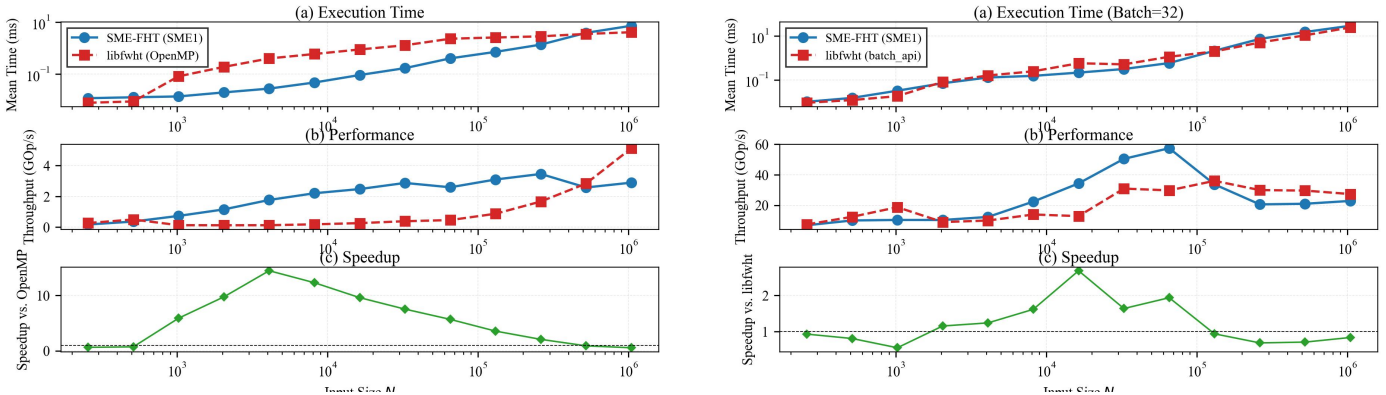
Q0 全 INT4 可节省 74.5%，但 Decode 开销增至 +3.49%

阶段结论

- **基线：** Prefill 1070.79 input tok/s，Decode 57.46 tok/s，后续比较沿用统一 backend 与统计口径。
- **主候选：** Q16 = 16 层 INT8 + 45 层 INT4，在容量收益和 Decode 开销之间最均衡。
- **收益：** DDR KV 从 1367.2 MB/rank 降至 438.6 MB/rank；Prefill / Decode 开销仅 +0.53% / +0.71%。

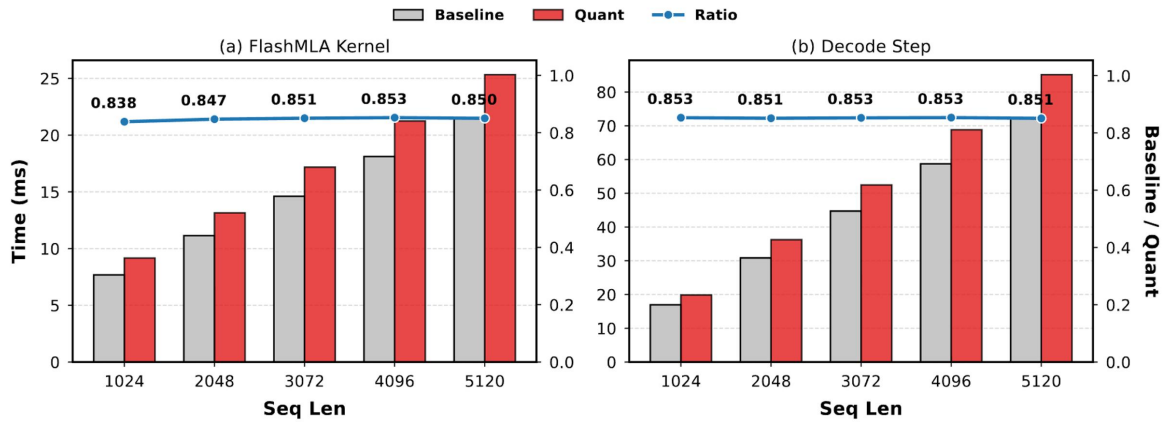
KVServe-Kunpeng：CPU KV Cache压缩的原生加速

FWHT变换算子



- 核心目标：将局部 Hadamard 子问题“原地”保留在 ZA 中完成
- 单batch情况最高可加速14x；32batch情况最高可加速1.73x

量化与解量化算子



单机Prefill阶段延迟

BF16	8bit	4bit	混合精度
1627ms	1658ms	1624ms	1624~1658ms

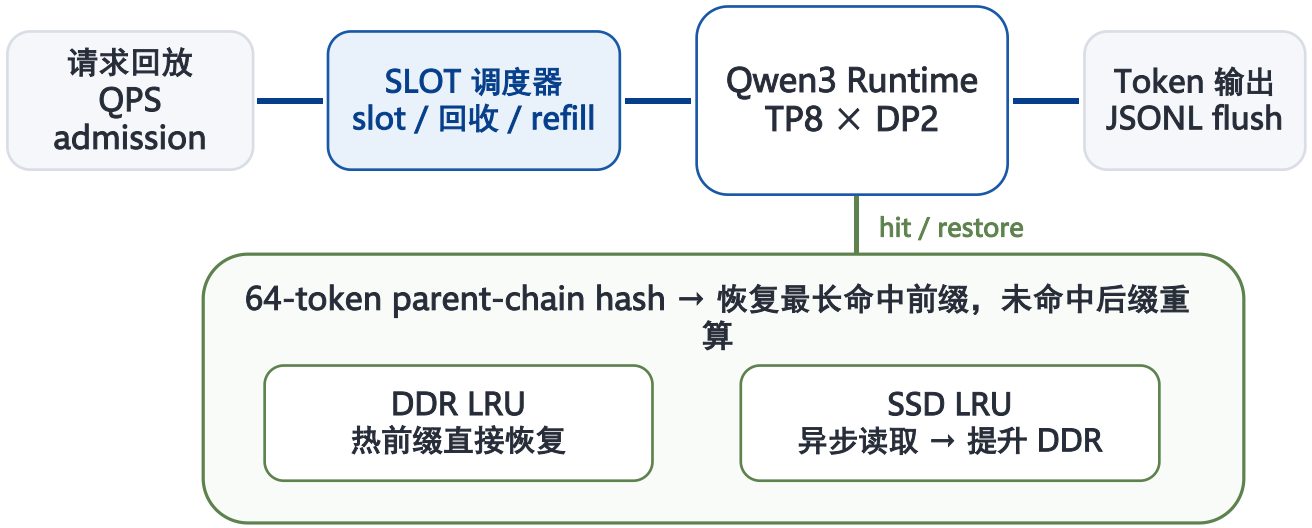
单机Decode阶段延迟

BF16	8bit	4bit	混合精度
1279ms	1255ms	1092ms	1092~1255ms

Kvserve-Kunpeng： 多级缓存前缀复用

- 已实现 Qwen3-14B 连续批处理与 DDR + SSD 多级缓存；14 请求中复用 33.9% prompt token，输出字节级一致
- 双路基线结果：打通 Prefill → DDR → Decode 量化链路，KV 缓存压力降低 67.9%，Decode 开销仅 +0.71%

Qwen3 + 连续批处理 + 多级缓存



实测结果

Prompt token 复用：33.9%

请求命中：6 hit / 8 miss = 42.9%

平均 Decode：117.1 → 114.6 ms（−2.1%）

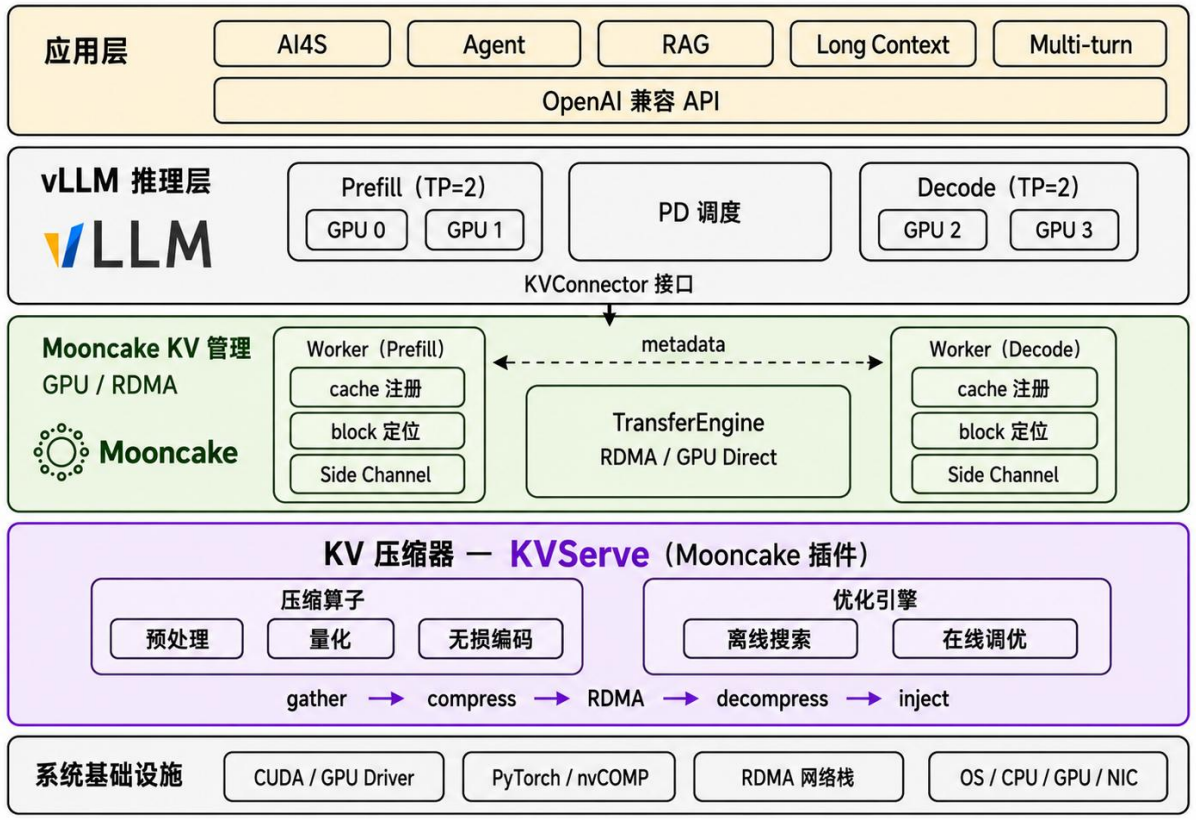
输出正确性：SHA-256 字节级一致

KVServe 与推理框架集成：vLLM+MoonCake

- KVServe 已基于 vLLM 0.31.0 打通 Mooncake P/D 压缩；可插拔 KV 压缩 RFC/PR 已提交上游（vLLM #59538 #61085）

vLLM 集成方案

作为灵活插件接入，配合Mooncake进行高效KV传输
典型应用场景：跨节点、跨集群 PD分离推理（Prefill-as-a-service）



1 接入 vLLM 0.31.0 Mooncake P/D（插件）

由外置实验 connector 转入 vLLM 解耦推理已广泛采用的 Mooncake 传输层；RFC 处于开放讨论，计划分两阶段进入上游主路径。

2 适配 混合注意力架构与共享缓存

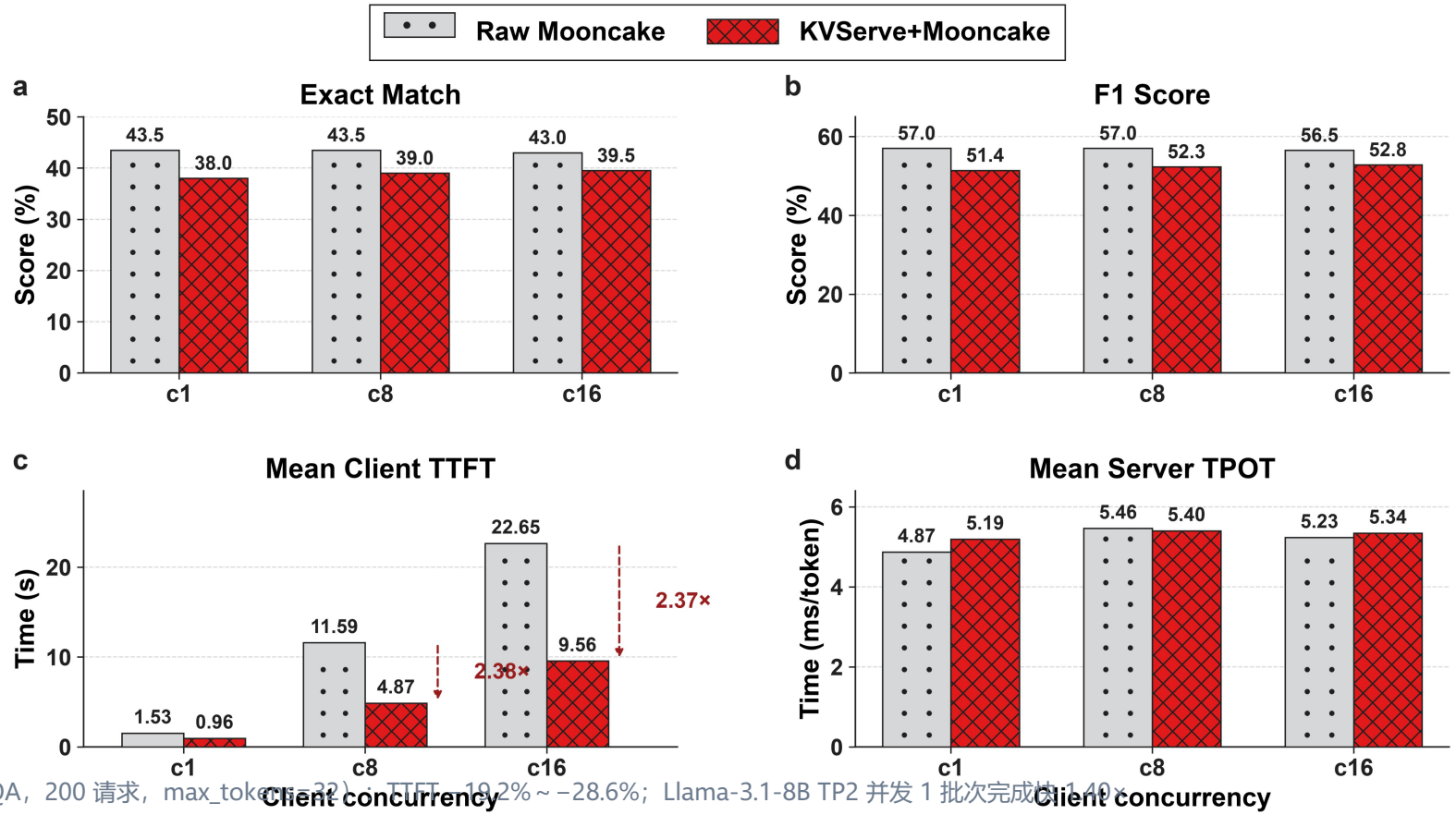
Full Attention 走压缩，Mamba/GDN 保持原始 RDMA，带宽用在真正占流量的 KV 页上。

3 压缩内嵌于 Mooncake 传输平面

以 Mooncake RDMA 与握手承载压缩流水，压缩成为 connector 上的一个阶段，而不是并行的第二套通信系统。

KVServe-MoonCake 端到端性能收益

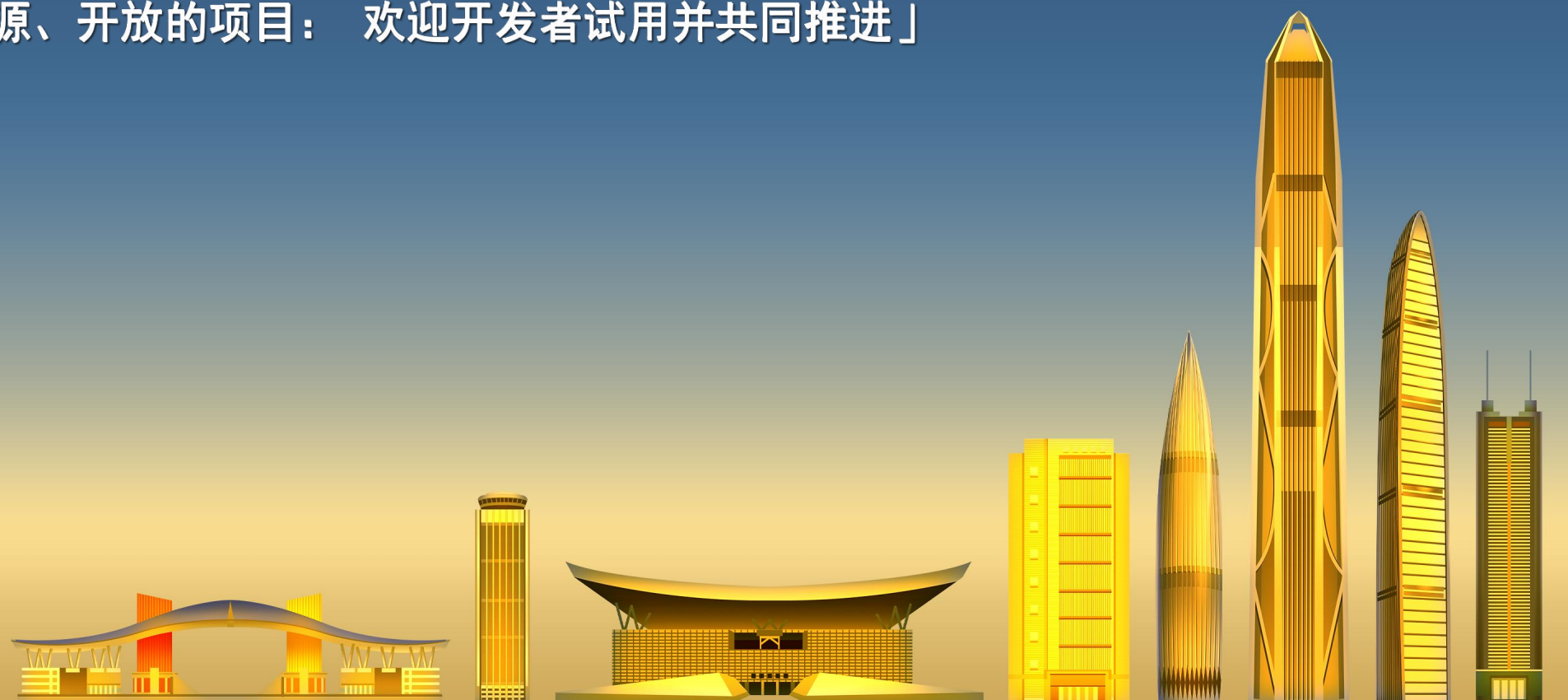
- 5090 双节点 · 50 Gb/s · vLLM 0.26.0 · Llama-3.1-8B · HotpotQA
- TTFT：并发 1 时 1.53 s → 0.96 s；并发 8 / 16 分别降至 4.87 s / 9.56 s，对应约 2.4× 的首 token 时延下降；服务端 TPOT 稳定在约 5.2–5.5 ms/token，与基线持平



0.30.0 + 200 Gb/s 复测（HotpotQA，200 请求，max_tokens=32）：TTFT −19.2% ~ −28.6%；Llama-3.1-8B TP2 并发 1 批次完成快 1.40×

04

「KVServe是一个开源、开放的项目： 欢迎开发者试用并共同推进」



扩展点：三个可替换阶段 + 一个统一的范式

压缩流水线由可插拔阶段组成，任意组合都能进入离线搜索空间

仓库路径：kvserve_v1/compression · kvserve_v1/connector



换掉一个阶段

进入离线搜索候选空间

与其余阶段自由组合

在线决策器自动选中

不划算时退回 raw 传输 —— 压缩是传输上的一个阶段，而不是并行的第二套通信系统

替换什么

变换算子

旋转 / 正交变换，改善量化时的可分布性；也可以留空，只做量化。

示例：Hadamard · 随机旋转

替换什么

量化器

按 Head 或 Layer 混合精度；实现 quantize / dequantize 即可接入。

示例：MixHQ · KIVI · CacheGen

替换什么

编码器

nvCOMP（ANS）与 LC 无损后端；新的无损后端可以继续加。

示例：nvCOMP ANS · LC codec

连接器契约：压缩内嵌于 vLLM / Mooncake connector，是传输上的一个阶段，而不是并行的第二套通信系统。

结果：算法实现、阶段组合、传输平面三者解耦 —— 换算法不需要改调度，换传输不需要改算法。

第一步 · 定义一个压缩器：两个方法就够

示例逐字来自仓库 examples/README.md（v1 分支）

examples/README.md · Custom Compression Component

my_quantizer.py

```python
# 只要实现 quantize / dequantize 两个方法
class MyQuantizer:
    def __init__(self, **kwargs):
        self.scale = kwargs.get("scale", 1.0)

    def update_params(self, **kwargs):
        self.scale = kwargs.get("scale", self.scale)

    def quantize(self, layer_id, tensor, **kwargs):
        # tensor: [2, num_blocks, block_size, num_kv_heads, head_size]
        meta = {"original_dtype": str(tensor.dtype).replace("torch.", "")}
        return tensor, meta

    def dequantize(self, layer_id, tensor, meta, **kwargs):
        return tensor
```

契约只有三件事：拿到一块 KV → 返回（压缩后张量，元数据）→ 能按元数据恢复；其余一切（缓存映射、跨层调度、传输）都由框架处理。

对应到代码：只有 quantize() 与 dequantize() 两个方法需要你写。

张量契约

单层

[2, num_blocks, block_size,
num_kv_heads, head_size]

全部层

[num_layers, 2, num_blocks, block_size,
num_kv_heads, head_size]

关键：作用域

- 只在 vLLM 托管的 KV block 上工作
- 不触碰 vLLM 调度器状态
- 无需改 KV block 管理与内存分配
- 本示例是恒等变换，仅演示契约

常见误用

别把压缩塞进 KV block 分配路径。

第二步 · 修改接入：注册进候选库

在 KVCompressionAdapter._build_manager() 里选择你的实现，再在压缩档里启用

examples/README.md · 无需改动 vLLM 调度与 KV block 管理

compression manager · 适配层

```
# 1) 注册：在 KVCompressionAdapter._build_manager() 中选择实现
if cfg_dict.get("quantizer_impl") == "my_quantizer":
    from my_package import MyQuantizer
    quantizer_cls = MyQuantizer

# 2) 启用：压缩档（也可经 kv_connector_extra_config 传入）
"compression": {
    "enabled": True,
    "pipeline": ["quantizer", "codec"],
    "quantizer_impl": "my_quantizer",
    "quantizer_config": {"scale": 1.0},
    "codec_config": {"codec_type": "nvcomp",
                     "nvcomp_algorithm": "ANS"},
}
```

接入后无需改动 **vLLM 调度与 KV block 管理**；在线决策器会把它当成一个普通候选，不划算时仍退回 raw 传输。

换句话说：你只扩展候选库，调度与内存管理保持原样。

改完之后自动获得什么

- 新算法进入离线搜索的候选空间
- 与 Transformer / Codec 自由组合
- 在线决策器可自动选中它
- 不划算时仍可退回 raw 传输

为什么只有这几行

压缩被收敛成一个窄接口：连接器负责缓存映射与请求协调，压缩框架负责组合与执行，算法方只提供实现。

提示

注册只改 _build_manager() 一处。

开发者可以从哪些点切入

仓库已提供 vLLM 连接器配置示例、组件扩展说明、最小自定义量化器示例

github.com/hpdps-group/KVServe

1 | 维护推理服务？

用真实负载帮我们验证：带宽 / SLO / 质量三重约束下的画像，比再加一个合成 benchmark 更有价值。欢迎提供 trace 片段。

2 | 研究压缩算法？

把新方法接进候选库：Transformer / Quantizer / Codec 三阶段可独立替换，自动进入离线搜索空间并被在线决策器选中。

3 | 熟悉 Kernel 或框架？

一起拓展执行后端与连接器：TileLang 融合算子、LC 无损后端、Mooncake Store 传输与 vLLM 连接器。

4 | 只想先跑起来？

仓库已含 vLLM 连接器配置示例、组件扩展说明与最小自定义量化器示例，5 分钟即可跑通。

5 分钟 Start up 例子

克隆仓库 → pip install -e . → 跑 tests/test_kvserve.py → 换掉一个示例量化器，即可看到差异。

扫码直达仓库

Start here · 参与指南

扫二维码，添加我为朋友。

hpdps-group/KVServe · 中科院计算所 刘泽栋

欢迎在 GOSIM 现场交流

GOSIM

Thanks!

GOSIM Shenzhen 2026

欢迎在 GOSIM 现场交流 · github.com/hpdps-group/KVServe

HPDPS 研究组 · 相关项目 同一套「压缩」思路在AI训推Infra的其他应用：通信 · 张量 · 权重

COCCL 压缩通信库

基于 NCCL 的可压缩集合通信库：压缩与精度协同感知，minmaxUint8 下集合通信约 3× 加速，支持各类聚合原语。

PPoPP 2026 · github.com/hpdps-group/COCCL

TACO 张量压缩

面向张量并行的中间张量压缩：自适应尺度 Hadamard 变换 + 双尺度量化，与 COCCL 协同，端到端吞吐最高 1.87×，精度近无损。

HPDC 2026 · arXiv:2604.24088

ENEC 权重压缩

面向昇腾 NPU 的无损模型权重压缩器：提供 NPU kernel 实现，以及压缩率与吞吐的测试工具链。

ISCA 2026 · github.com/hpdps-group/ENEC

KVServe · COCCL · TACO · ENEC 均出自 HPDPS 研究组（中国科学院计算技术研究所 · 陶鼎文研究员领导）

